

Computing for Analytical Work

Session 3 — What broke?

Block 5 — Reading errors and debugging systematically

Knowledge check 3 — 10 minutes, closed book, 3 questions

- Covers **Session 2 and Homework 2**: interpreters, environments, `uv`, notebook state, Git checkpoints
- Short answers, in your own words
- Diagnostic, not tricky — if you did the work, this is easy
- Closed book means closed: no laptop, no phone, no notes
- **This block's lecture is 35 minutes**, not 45 — the check comes out of lecture, never out of lab
- Hand it in when the timer ends; the lecture starts immediately after

This block in one sentence

Error messages are evidence. Debugging is a method, not a vibe.

A traceback is not a punishment. It is the machine telling you, precisely, what it could not do and where it gave up.

What we cover

1. What a traceback is, and the two anchors: *what* went wrong, and *where* in your code
2. Symptom versus cause
3. Six errors you will meet this year, and one prediction
4. Inspect before you guess: print, minimal example, asking well
5. Git thread — `git log` : what happened before?
6. AI thread — a prompt made of evidence
7. Lab 5 — The traceback is the map

What a traceback is

- Python hit something it could not do, and stopped
- On its way out, it wrote down the **chain of calls** that was active when it failed
- It is printed **top to bottom**: the outermost call first, the failure last
- That is why the useful part is at the **bottom**
- Nothing about it is random; every line points at a file and a line number

First: find it in the scroll

- Today every run is a script **in a terminal** — and a terminal keeps scrolling after an error
- The traceback begins at the literal line `Traceback (most recent call last):` — that is what you search for
- Scroll back: trackpad, scrollbar, or **Cmd-F / Ctrl-F in VS Code's terminal**
- Several runs' output stacked up? You cannot tell which error is *this* run's
- The habit that removes the problem: **clear** before every rerun — then everything on screen is from this run
- One traceback can be taller than your window. The **bottom** is still where you start

A real one

```
anna@laptop project $ uv run python scripts/report.py
Traceback (most recent call last):
  File "/Users/Shared/anna/project/.venv/lib/python3.13/site-packages/pandas/core/indexes/base.py", line 3641, in get_loc
    return self._engine.get_loc(casted_key)
           ~~~~~~
  File "pandas/_libs/index.pyx", line 168, in pandas._libs.index.IndexEngine.get_loc
  File "pandas/_libs/index.pyx", line 197, in pandas._libs.index.IndexEngine.get_loc
  File "pandas/_libs/hashtable_class_helper.pxi", line 7668, in pandas._libs.hashtable.PyObjectHashTable.get_item
  File "pandas/_libs/hashtable_class_helper.pxi", line 7676, in pandas._libs.hashtable.PyObjectHashTable.get_item
KeyError: 'revenue'
```

The above exception was the direct cause of the following exception:

```
Traceback (most recent call last):
  File "/Users/Shared/anna/project/scripts/report.py", line 9, in <module>
    total = monthly_total(df)
  File "/Users/Shared/anna/project/scripts/report.py", line 5, in monthly_total
    return df["revenue"].sum()
           ~~~~~~
  File "/Users/Shared/anna/project/.venv/lib/python3.13/site-packages/pandas/core/frame.py", line 4378, in __getitem__
    indexer = self.columns.get_loc(key)
  File "/Users/Shared/anna/project/.venv/lib/python3.13/site-packages/pandas/core/indexes/base.py", line 3648, in get_loc
    raise KeyError(key) from err
KeyError: 'revenue'
```

The two anchors

1. **Last line** → the **error type** → *what kind of problem*
2. **Nearest line above that is in YOUR file** → the **failing line** → *where*

Everything else is the call stack. Scroll up later, if the two anchors are not enough.

The same traceback, annotated

[an earlier traceback, 5 frames inside pandas, ends in the same KeyError – chained; skip it]
 Traceback (most recent call last):
 File `"/Users/Shared/anna/project/scripts/report.py"`, line 9, in `<module>`
 total = monthly_total(df)
 2 File `"/Users/Shared/anna/project/scripts/report.py"`, line 5, in `monthly_total`
 return df["revenue"].sum()
     ~~~~~  
 File `"../site-packages/pandas/core/frame.py"`, line 4378, in `__getitem__`  
     indexer = self.columns.get\_loc(key)  
 File `"../site-packages/pandas/core/indexes/base.py"`, line 3648, in `get_loc`  
     raise KeyError(key) from err  
 1 **KeyError: 'revenue'**

read upward ↑

3

- 1 **What: the last line**  
the error type, and the value it could not find
- 2 **Where: nearest line above**  
that is in a file you wrote, and the code on that line
- 3 **The call stack**  
frames in site-packages belong to pandas; the bug is almost never there

**Everything else:**  
**second-tier evidence.**

Long paths are shortened at ...

- **What:** `KeyError: 'revenue'` — a name was looked up and did not exist
- **Where:** `scripts/report.py`, line 5, inside `monthly_total: df["revenue"].sum()`
- **Next step, before any change:** `print(df.columns)` — which column names *does* it have?

## When pandas is in the way

- Each `File "...", line ...` entry is a **frame**: one call in the chain
- Frames in `site-packages/` belong to pandas and the packages it uses — **not to you**
- The bug is almost never inside pandas
- Walk **up** from the bottom until the path is a file in **your** project
- That frame is anchor 2, even if it is six lines above the error
- **Two tracebacks, joined by a sentence?** Read the last one; the first is the same failure seen from inside pandas
- Notebook tracebacks name a **cell** instead of a file ( `Cell In[3], line 2` ) and obey the same rule

## Symptom versus cause

- **Symptom:** what the traceback says — `KeyError: 'revenue'`
- **Cause:** why the data looks like that — the CSV header says `Revenue`, capital R, and the line that renamed it was deleted
- The traceback gives you the symptom for free; the cause you have to go and find
- Fixing the symptom: `df["Revenue"]` — works until the next file
- Fixing the cause: rename once, right after loading, and say so
- The diagnosis note asks for both, on purpose

## **FileNotFoundError** and **ModuleNotFoundError**

**FileNotFoundError: [Errno 2] No such file or directory: 'data/sales.csv'**

- Usually means: the path is relative to the **wrong working directory**, or the file is called something else
- Inspect first: `pwd` , `ls data/` , `os.getcwd()` in the notebook

**ModuleNotFoundError: No module named 'pandas'**

- Usually means: the Python that ran this **does not have it** — you are in a different **environment** from the one the package was installed into, or the project never declared it
- Inspect first: `sys.executable` from the process that failed, then `pyproject.toml`

## **NameError** and **KeyError**

### **NameError: name 'df' is not defined**

- Usually means: the line that creates `df` never ran — typo, skipped cell, wrong order
- Inspect first: search the file for `df =`; in a notebook, Restart, then Run All

### **KeyError: 'revenue'**

- Usually means: a column, dictionary key, or config entry is **spelled differently** than you think — or is created by a step that has not run yet
- Inspect first: `print(df.columns.tolist())` or `print(d.keys())`

## Predict: which error?

```
float("€1,200")
```

```
3 + "3"
```

Two one-liners. Each raises an error. **Which type does each raise?** Write down two names. Do not type.

## Reveal: `ValueError` and `TypeError`

`ValueError: could not convert string to float: '€1,200'`

- The **type was acceptable** — a string can become a float — but **this value** cannot: there is a euro sign in it
- Inspect first: look at the actual values — `print(df["amount"].head())`,  
`print(df["amount"].unique()[:10])`

`TypeError: unsupported operand type(s) for +: 'int' and 'str'`

- The **kind of thing is wrong** — no string can be added to an integer, whatever it contains
- Inspect first: `print(type(x))`, `print(df.dtypes)`

## Print, log, inspect — before guessing

- Guessing is editing code you have not looked at, to fix data you have not looked at
- One `print()` right above the failing line beats ten minutes of theorising
- Print the thing the line uses: `print(df.columns)`, `print(type(x))`, `print(path.exists())`
- Then run the whole script again, with the **same command** that failed — `uv run python scripts/report.py` — and expect the **same error**, with your print's output just above it
- Remove the print when done; the diagnosis note is where the finding lives, and `git diff` will show you the print you forgot



## Minimal reproducible example

```
import pandas as pd
df = pd.read_csv("data/raw/sales.csv")
print(df.columns.tolist())
print(df["revenue"].sum())      # the line that fails, isolated
```

```
['date', 'Revenue', 'region']
Traceback (most recent call last):
...
KeyError: 'revenue'
```

- Strip everything that is not needed to produce the error
- If four lines still fail, the bug is in those four lines
- If four lines work, the bug is in what you removed — add pieces back until it breaks
- This is also the best thing to paste when asking for help

## How to ask for help well

A question a TA, a colleague, or an assistant can actually answer contains:

1. The **exact** error, pasted, not paraphrased
2. The **failing line** and the file it lives in
3. Where things are: file tree, working directory, the command you ran
4. What you **expected** and what **happened** instead
5. What you **already tried** — and what it showed you

"It doesn't work" contains none of these.

## **Git thread — what happened before?**

## `git log` — the history of your checkpoints

```
git log --oneline
```

- One line per commit, newest first: a **hash** — the commit's ID, what you hand to `git diff` — and the message you wrote
- Every commit is a moment the project was in a known state
- Small commits mean small distances between *working* and *broken*, and small distances are easy to search
- Lab 5 asks for **one commit per fix**: by the end, your own log is four checkpoints deep

# If it worked before, history tells you what changed

```
anna@laptop project $ git log --oneline -- scripts/report.py
449ac0d (HEAD -> main) Tidy report.py: drop the rename step
911cfdb Add monthly_total()
15f0507 Initial working report
anna@laptop project $ git diff 911cfdb -- scripts/report.py
diff --git a/scripts/report.py b/scripts/report.py
index 35b2cdc..96296ea 100644
--- a/scripts/report.py
+++ b/scripts/report.py
@@ -6,6 +6,5 @@ def monthly_total(df):

    df = pd.read_csv("data/raw/sales.csv")
-   df = df.rename(columns={"Revenue": "revenue"})
    total = monthly_total(df)
    print(total)
anna@laptop project $ █
```

- `git log --oneline -- scripts/report.py` — only the commits that touched *this* file: the suspect list
- `git diff <hash> -- scripts/report.py` — everything that changed since the commit you name
- You are not guessing what you changed; you are **reading** it

## **AI thread — a prompt made of evidence**

## "Fix this" versus evidence

### Bad prompt

```
my script doesn't work fix this
```

### Good prompt

```
Running `uv run python scripts/report.py` from the project root gives  
KeyError: 'revenue' at scripts/report.py line 5: df["revenue"].sum().  
print(df.columns.tolist()) shows ['date', 'Revenue', 'region'].  
I expected a column named 'revenue'. I have not changed the CSV.  
What are the most likely causes, and what should I inspect next?
```

## AI gets dramatically better when you provide evidence instead of frustration

- The five ingredients from *asking well* are the prompt
- Ask for **likely causes and what to inspect**, not "fix it"
- Try **one** change; run the **same** command; read the result yourself
- Then explain the fix in your own words — that sentence goes in the diagnosis note
- Your AI-use note names what you asked and how you verified the answer



## **5-minute stand-and-stretch**

Stand up. Leave the laptop. Back in five for Lab 5.

## Lab 5 — The traceback is the map

**First 15 minutes: no AI.** The traceback and the failing file only.

- Chat windows, AI editors, assistant tabs: **closed**. Search engines are fine
- Why: the reflex we are installing is *"check the columns"*, not *"ask what to check"*
- Only one of those reflexes survives a closed-book exam
- Solve it early? Write the notes and commit; the pair window is at minute 33
- After 15 minutes, normal AI policy resumes. Nobody is penalised; you will be asked to close a window, that is all

## Lab 5 — get it, run it, read the bottom

In your terminal (Git Bash on Windows, Terminal on macOS), in the folder where you keep course work:

```
git clone https://github.com/earino/ecbs5293-lab05-tracebacks.git
cd ecbs5293-lab05-tracebacks
uv sync
uv run python scripts/report.py
```

- Open the **folder** in VS Code (trust it if asked) to edit
- `scripts/report.py` fails **four times**, each of a different kind: a filename, a column name, a value, a type. Each fix reveals the next
- Read the two anchors. Inspect. Change **one** thing. Run the same command again

```
n3.13/site-packages/pandas/io/parsers/readers.py", line 1645, in __i
nit__
    self._engine = self._make_engine(f, self.engine)
                        ~~~~~^~~~~~
File "/Users/Shared/anna/ecbs5293-lab05-tracebacks/.venv/lib/pytho
n3.13/site-packages/pandas/io/parsers/readers.py", line 1904, in _ma
ke_engine
 self.handles = get_handle(
                    ~~~~~^
                        f,
                        ^^
    ...<6 lines>...
                        storage_options=self.options.get("storage_options", None),
                        ~~~~~^~~~~~
)
 ^
File "/Users/Shared/anna/ecbs5293-lab05-tracebacks/.venv/lib/pytho
n3.13/site-packages/pandas/io/common.py", line 930, in get_handle
 handle = open(
 handle,
 ...<3 lines>...
 newline="",
)
FileNotFoundError: [Errno 2] No such file or directory: 'data/raw/or
ders.csv'
anna@laptop ecbs5293-lab05-tracebacks $
```

## What you produce, and the last ten minutes

1. The script running end to end: a table by status and a `Total:` line
  2. **One commit per fix, four in all** — `git add .`, `git commit -m "<what you fixed>"`; `git log --oneline` shows your four above the starter's one
  3. `DIAGNOSIS.md` — **one five-part note per failure**, four sections, laid out for you. Part 3: the two anchors, **pasted before you change anything**, plus the one inspection that showed the cause
  4. **Minute 33**: explain one of the four to a neighbour, about a minute, no notes; your neighbour picks which. The what-if is on the slide that goes up then
  5. `DIAGNOSIS.md` to the Lab 5 checkpoint on Moodle before you leave
- Stretch: one `assert` per stage with a message that would have caught it earlier; a three-line minimal example for the value failure. Resets in the README; both destroy work.

## The last ten minutes — explain it to a neighbour

**Speaker, about a minute, pointing at the screen, not reading the note:** symptom → cause → evidence → change → verification.

**Listener asks:**

1. Show me the evidence.
2. Why did it fail — not just where?
3. Undo your first fix, in your head only, and run again. Which of the four errors do you see, and do the other three still exist?

Swap. Unsure, or you disagree? **Hand up.** Then the answer, for everyone. Before you leave: `DIAGNOSIS.md` to the Lab 5 checkpoint on Moodle.